TRIBHUVAN UNIVERSITY

INSTITUTE OF ENGINEERING

PULCHOWK CAMPUS

A LAB REPORT
ON
Student Management System
(Django Applications)

Submitted To:
Prakash Chandra
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING
PULCHOWK CAMPUS, LALITPUR

Submitted By:
Abhishek Tharu (PUL080BCT008)

1. Project Description

1.1 Overview

The Django Library Management System is a full-stack web application built to demonstrate core Django concepts including the MVT architecture, ORM-based database integration, user authentication and authorization, session management, custom middleware, and RESTful routing.

1.2 Key Features

- Book CRUD — Create, Read, Update, Delete books with genre associations
- User Authentication — Registration, login, logout with session cookies
- Search & Filter — Filter books by title, author, and availability
- Session-based Reading List — Tracks user's reading list across requests
- Custom Middleware — Request logging, IP blocking, and error handling
- Django Admin — Full admin panel for superusers
- Relational DB (SQLite) + NoSQL concept illustration (MongoDB)

1.3 Technologies Used

Technology	Purpose
`Python 3.x + Django 4.x`	Core backend framework
`SQLite`	Relational database (default)
`MongoDB + pymongo`	NoSQL illustration
`Bootstrap 5`	Frontend styling
`django-crispy-forms`	Form rendering
`django-widget-tweaks`	Template form customization

2. MVC / MVT Architecture

2.1 Django MVT vs Classical MVC

Django follows the MVT (Model-View-Template) pattern, which is Django's variant of the classic MVC (Model-View-Controller) pattern. The difference is that Django's 'View' acts as the Controller, and 'Template' acts as the View layer.

MVC Role	MVT Equivalent	Responsibility in This Project
Model	Model	Book, Genre, ReadingList ORM classes
View	Template	HTML templates in /templates/
Controller	View	Functions in library/views.py
Router	URL Conf	library/urls.py, accounts/urls.py
Middleware	Middleware	core/middleware.py (logging, IP block)

2.2 Request-Response Flow

The flow of a request through Django's MVT architecture follows this path:

```
Browser Request
     |
     v
  Django URL Dispatcher (urls.py)
     |   -- matches URL pattern
     v
  Middleware Stack (core/middleware.py)
     |   -- logs request, checks IP
     v
  View Function (library/views.py)
     |   -- handles business logic
     |   -- queries Model (ORM)
     v
  Template Engine (.html file)
     |   -- renders HTML with context
     v
  HTTP Response --> Browser
```

3. Code: Major Components

3.1 models.py — Database Models (ORM)

Defines the database schema using Python classes. Each class maps to a database table.

```
# library/models.py
from django.db import models
from django.contrib.auth.models import User

class Genre(models.Model):
    name = models.CharField(max_length=100, unique=True)
    def __str__(self): return self.name

class Book(models.Model):
    title       = models.CharField(max_length=200)
    author      = models.CharField(max_length=150)
    isbn        = models.CharField(max_length=13, unique=True)
    published   = models.DateField()
    genre       = models.ForeignKey(Genre, on_delete=models.SET_NULL, null=True)
    description = models.TextField(blank=True)
    available   = models.BooleanField(default=True)
    added_by    = models.ForeignKey(User, on_delete=models.SET_NULL, null=True)
    created_at  = models.DateTimeField(auto_now_add=True)

    def __str__(self):
        return f'{self.title} by {self.author}'

class ReadingList(models.Model):
    user  = models.OneToOneField(User, on_delete=models.CASCADE)
    books = models.ManyToManyField(Book, blank=True)
```

3.2 urls.py — URL Routing

Maps URL patterns to view functions. Named URLs allow use of {% url 'name' %} in templates.

```
# core/urls.py (project-level)
```

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('library.urls')),
    path('accounts/', include('accounts.urls')),
]

# library/urls.py (app-level)
from django.urls import path
from . import views

urlpatterns = [
    path('',                      views.book_list,   name='book_list'),
    path('book/<int:pk>/',        views.book_detail, name='book_detail'),
    path('book/add/',             views.book_create, name='book_create'),
    path('book/<int:pk>/edit/',   views.book_update, name='book_update'),
    path('book/<int:pk>/delete/',views.book_delete, name='book_delete'),
]
```

3.3 views.py — View Functions (Controller)

Handles HTTP requests and responses, applies business logic, queries the database via ORM, and returns rendered templates.

```
# library/views.py
from django.shortcuts import render, get_object_or_404, redirect
from django.contrib.auth.decorators import login_required
from django.contrib import messages
from .models import Book, ReadingList
from .forms import BookForm, SearchForm

def book_list(request):
    form  = SearchForm(request.GET)
    books = Book.objects.select_related('genre').all()
    if form.is_valid():
        q = form.cleaned_data.get('query')
        if q:
            books = books.filter(title__icontains=q) \
                  | books.filter(author__icontains=q)
    # Session: track visit count
```

```
    request.session['visit_count'] = \
        request.session.get('visit_count', 0) + 1
    return render(request, 'library/book_list.html', {
        'books': books, 'form': form,
        'visit_count': request.session['visit_count']
    })

@login_required
def book_create(request):
    if request.method == 'POST':
        form = BookForm(request.POST)
        if form.is_valid():
            book = form.save(commit=False)
            book.added_by = request.user
            book.save()
            messages.success(request, f'"{book.title}" added!')
            return redirect('book_detail', pk=book.pk)
    else:
        form = BookForm()
    return render(request, 'library/book_form.html',
                  {'form': form, 'action': 'Add'})
```

3.4 forms.py — Forms & Validation

Defines ModelForm and plain Form classes with built-in and custom validation.

```
# library/forms.py
from django import forms
from .models import Book

class BookForm(forms.ModelForm):
    class Meta:
        model  = Book
        fields = ['title','author','isbn','published','genre','description']
        widgets = {
            'published': forms.DateInput(attrs={'type': 'date'}),
        }

    def clean_isbn(self):         # custom field validation
        isbn = self.cleaned_data['isbn']
        if not isbn.isdigit():
            raise forms.ValidationError('ISBN must be numeric.')
```

```
        return isbn

class SearchForm(forms.Form):
    query = forms.CharField(max_length=100, required=False,
        widget=forms.TextInput(attrs={'placeholder': 'Search...'}))
    available_only = forms.BooleanField(required=False)
```

3.5 middleware.py — Custom Middleware

Middleware sits between the web server and views, processing every request and response globally.

```
# core/middleware.py
import logging, time
logger = logging.getLogger(__name__)

class RequestLogMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        start = time.time()
        logger.info(f'-> {request.method} {request.path}')
        response = self.get_response(request)
        ms = (time.time() - start) * 1000
        logger.info(f'<- {response.status_code} | {ms:.1f}ms')
        return response

BANNED_IPS = ['192.168.1.100']

class BlockBannedIPMiddleware:
    def __init__(self, get_response):
        self.get_response = get_response

    def __call__(self, request):
        ip = request.META.get('REMOTE_ADDR')
        if ip in BANNED_IPS:
            from django.http import HttpResponseForbidden
            return HttpResponseForbidden('Access denied.')
        return self.get_response(request)
```

3.6 Authentication Views (accounts/views.py)

Handles user registration, login and logout. Django's login() function automatically creates a session and sets the sessionid cookie.

```
# accounts/views.py
from django.contrib.auth import login, logout
from django.contrib.auth.forms import UserCreationForm, AuthenticationForm
from django.shortcuts import render, redirect
from django.contrib import messages

def register_view(request):
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            user = form.save()
            login(request, user)   # creates session cookie
            messages.success(request, f'Welcome, {user.username}!')
            return redirect('book_list')
    else:
        form = UserCreationForm()
    return render(request, 'accounts/register.html', {'form': form})

def login_view(request):
    if request.method == 'POST':
        form = AuthenticationForm(data=request.POST)
        if form.is_valid():
            login(request, form.get_user())
            return redirect(request.GET.get('next', 'book_list'))
    else:
        form = AuthenticationForm()
    return render(request, 'accounts/login.html', {'form': form})

def logout_view(request):
    logout(request)   # clears session and cookie
    return redirect('login')
```

4. GitHub Repository

The complete source code for this project is available on GitHub. The repository includes all Django apps, templates, static files, requirements.txt, and setup instructions in the README.

Repository URL: [Click here]

4.1 Project Directory Structure

```
django_demo/
├── core/
│   ├── settings.py
│   ├── urls.py
│   └── middleware.py
├── library/
│   ├── models.py
│   ├── views.py
│   ├── forms.py
│   ├── urls.py
│   └── admin.py
├── accounts/
│   ├── views.py
│   └── urls.py
├── templates/
│   ├── base.html
│   ├── library/
│   │   ├── book_list.html
│   │   ├── book_detail.html
│   │   └── book_form.html
│   └── accounts/
│       ├── login.html
│       └── register.html
├── requirements.txt
├── manage.py
└── README.md
```

4.2 Setup Instructions

Run the following commands to set up and run the project locally:

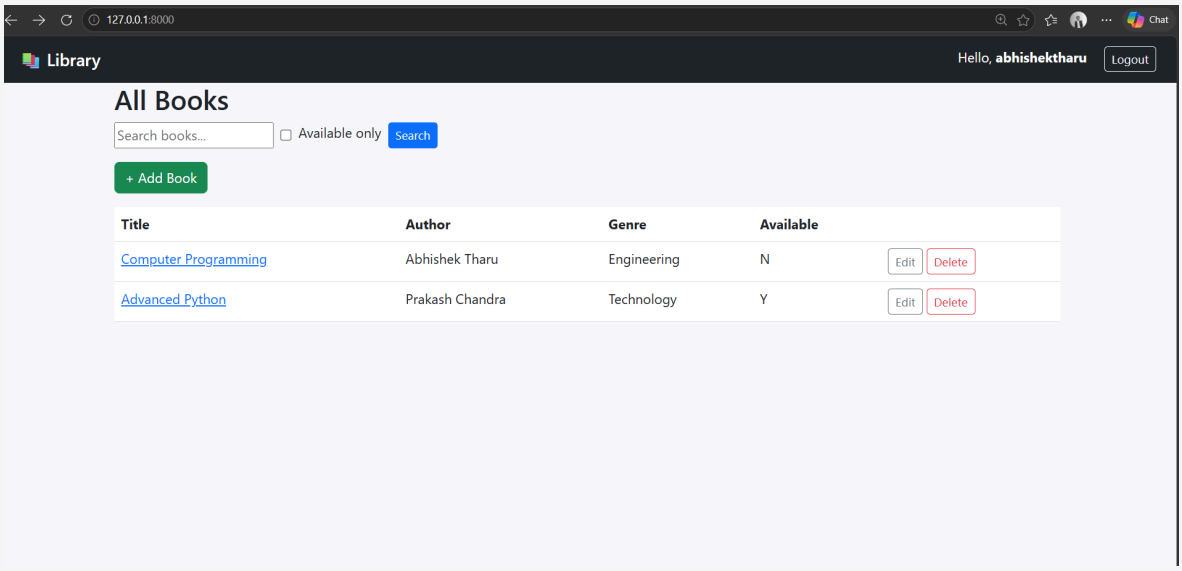
```
git clone https://github.com/YOUR-USERNAME/django-library-demo.git
cd django-library-demo
python -m venv venv && source venv/bin/activate
pip install -r requirements.txt
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
# Open http://127.0.0.1:8000 in your browser
```

5. Output Screenshots

The following section describes the expected output screens of the application. Replace the placeholder boxes below with actual screenshots from your running application.

5.1 Book List Page (Home)

URL: http://127.0.0.1:8000/

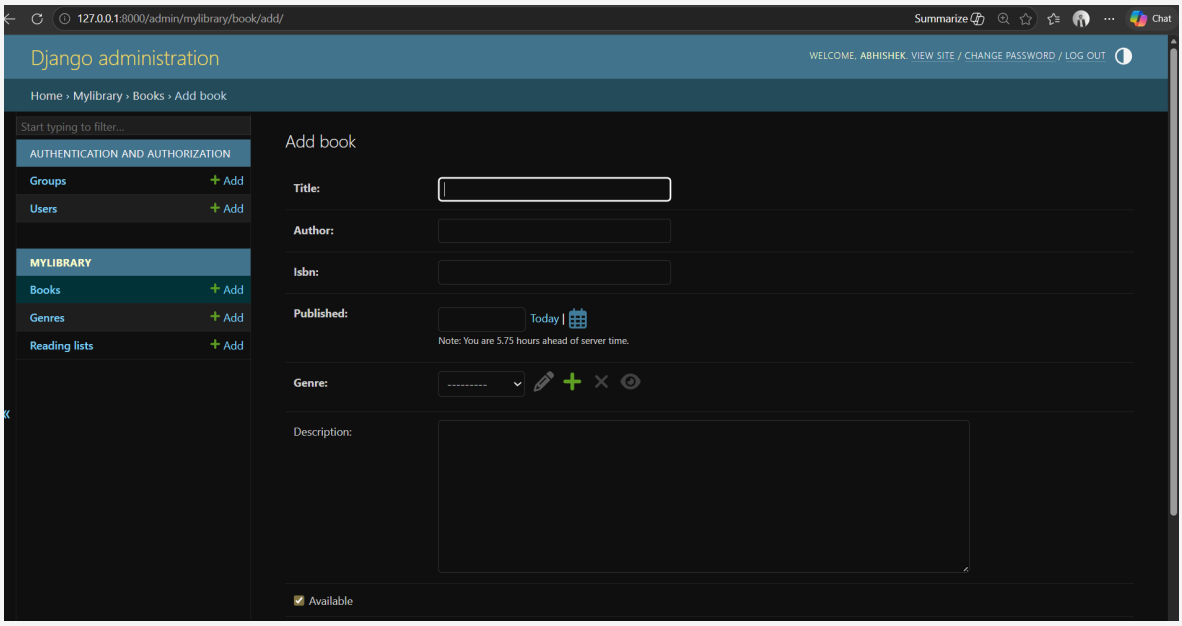


Book list

Displays all books with search/filter form. Session visit count shown in the heading. Add Book button visible for logged-in users.

5.2 Add Book Form

URL: http://127.0.0.1:8000/book/add/

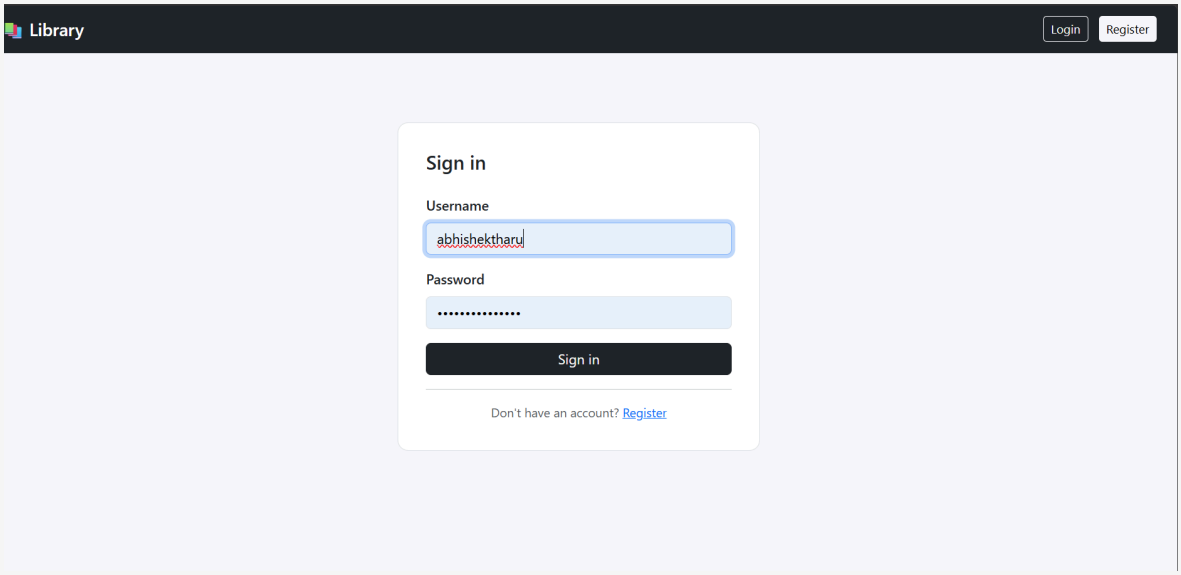


Add Book

ModelForm with all Book fields. Shows validation errors inline. Requires authentication — redirects to login if not logged in.

5.3 User Login Page

URL: http://127.0.0.1:8000/accounts/login/

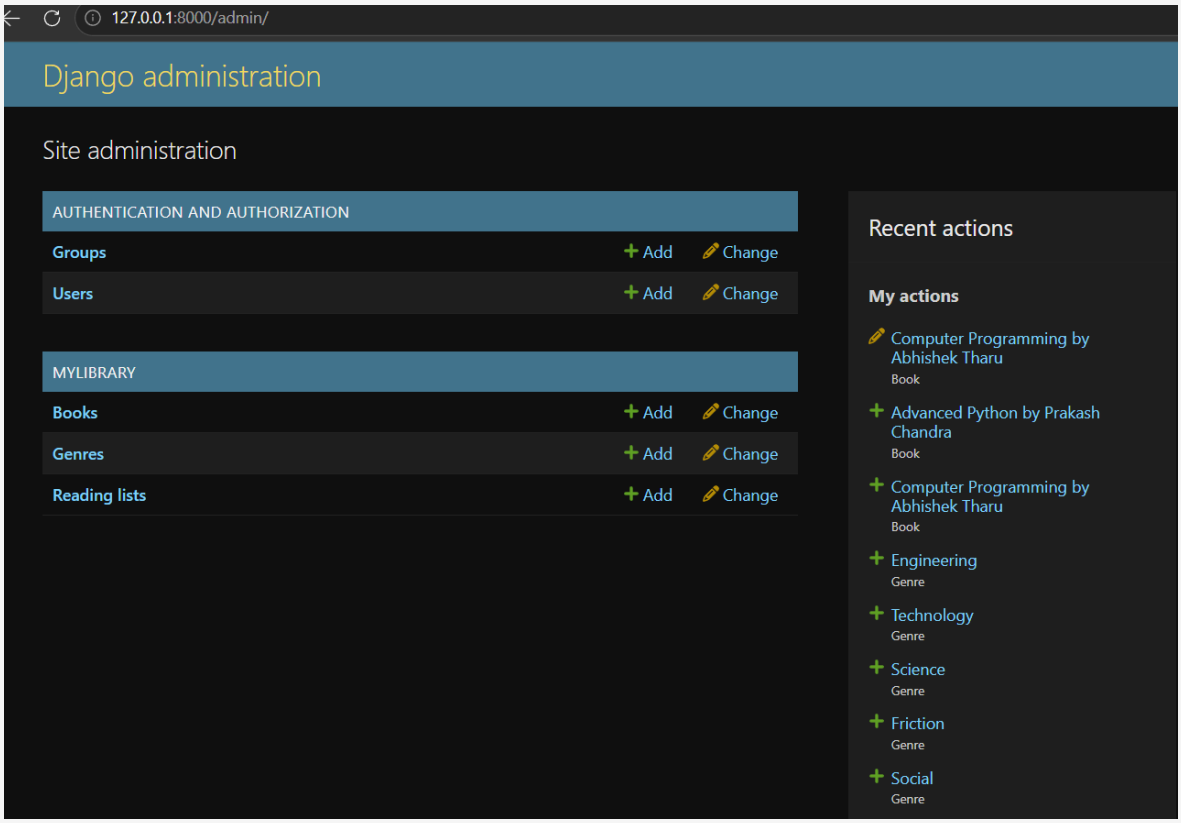


Login Page

Login form with username and password. Non-field errors shown in red alert box for wrong credentials.

5.4 Django Admin Panel

URL: http://127.0.0.1:8000/admin/



Admin Panel

Superuser admin interface. Lists Books, Genres, and ReadingLists with search, filter, and bulk actions.

6. Key Concepts Summary

Concept	Where it is in the project
Request/Response	`All views — request object, return render()/redirect()`
GET vs POST handling	`book_list() search form, book_create() POST`
Sessions	`request.session['visit_count'], toggle_reading_list()`
Cookies	`Set by login() via Django's sessionid cookie`
Routing	`urls.py in library/ and accounts/ apps`
Middleware	`core/middleware.py — logging, IP block, error`
Templating	`{% extends %}, {% url %}, {% if %}, {{ var }}`
ORM / Relational DB	`Book.objects.filter(), ForeignKey, ManyToMany`
NoSQL Concept	`nosql_example.py — pymongo CRUD illustration`
Authentication	`login(), logout(), @login_required decorator`
Authorization	`@login_required, request.user checks in views`
Form Validation	`BookForm.clean_isbn(), form.is_valid()`
CRUD Operations	`book_create, book_update, book_delete, book_list`